

CRUD Flow Tree — React vs Vue vs Angular (StreetScout)

Each branch/leaf is annotated with exact code snippets per framework. Rooted at App boot.

```
ROOT: App boot
■ Load OpenAPI spec (once)
  ■ React
  ■ Vue
  ■ Angular

■ Discover resources from spec (pure function)
  ■ React
  ■ Vue
  ■ Angular

■ Render per Resource (Street, Sign, Intersection)
  ■ React
  ■ Vue
  ■ Angular

■ List fetch (page/size/filters)
  ■ React
  ■ Vue
  ■ Angular

■ Create Street (repeat > page size)
  ■ React submit
  ■ Vue submit
  ■ Angular submit

■ Edit Street
  ■ React pick + submit
  ■ Vue pick + submit
  ■ Angular pick + submit

■ Create Sign (optionally filtered by streetId)
  ■ React submit
  ■ Vue submit
  ■ Angular submit

■ Edit Sign (detect move streetId A→B)
  ■ React onMoved
  ■ Vue onMoved
  ■ Angular onMoved

■ Create Intersection
  ■ React submit
  ■ Vue submit
  ■ Angular submit

■ Edit Intersection
  ■ React submit
  ■ Vue submit
  ■ Angular submit

■ Reconciliation / change detection aids
  ■ React keys & form key
  ■ Vue :key & v-model
  ■ Angular trackBy & OnPush
```

Load OpenAPI spec (once)

```
// React (App root)
function SpecProvider({ url, children }) {
  const [spec, setSpec] = React.useState(null);
  React.useEffect(() => {
    let canceled = false;
    (async () => {
      const res = await fetch(url);
      if (!canceled) setSpec(await res.json());
    })();
    return () => { canceled = true; };
  }, [url]);
  return <SpecCtx.Provider value={spec}>{children}</SpecCtx.Provider>;
}
// Vue 3 (Composition API)
```

```

import { ref, onMounted } from 'vue';
export function useSpec(url) {
  const spec = ref(null);
  onMounted(async () => {
    const res = await fetch(url);
    spec.value = await res.json();
  });
  return { spec };
}
// Angular (service + component)
@Injectable({ providedIn: 'root' })
export class SpecService {
  constructor(private http: HttpClient) {}
  getSpec(url: string): Observable<any> { return this.http.get(url); }
}

@Component({ /* ... */ })
export class AppComponent implements OnInit {
  spec$: Observable<any>;
  constructor(private specSvc: SpecService) {}
  ngOnInit() { this.spec$ = this.specSvc.getSpec('/api.json'); }
}

```

Discover resources from spec (pure function)

```

// React (shared util)
export function discoverResources(openapi) {
  if (!openapi?.paths) return [];
  // ... your byName assembly ...
  return Array.from(byName.values()).filter(looksApiLike);
}

const resources = React.useMemo(() => discoverResources(spec), [spec]);
// Vue (pure util + computed)
export function discoverResources(openapi) { /* pure */ }
const resources = computed(() => spec.value ? discoverResources(spec.value) : []);
// Angular (pure util + map())
discoverResources(openapi: any) { /* pure */ }

this.resources$ = this.spec$.pipe(
  map(spec => this.discoverResources(spec))
);

```

Render per Resource (Street, Sign, Intersection)

```

// React
return (
  <main>
    {resources.map(r => (
      <ResourceSection key={r.name} res={r} />
    ))}
  </main>
);

```

```

    (async () => {
      const data = await api.list(res, params);
      if (!canceled) setState(data);
    })();
    return () => { canceled = true; };
  }, [api, res.name, key]);
  return state;
}
// Vue 3
const page = ref(0), size = ref(20), filters = ref({});
const items = ref([]), total = ref(0);
watchEffect(async () => {
  const data = await api.list(res, { page: page.value, size: size.value, filters: filters.value });
  items.value = data.items; total.value = data.total;
});
// Angular (RxJS)
page$ = new BehaviorSubject(0);
size$ = new BehaviorSubject(20);
filters$ = new BehaviorSubject({});

items$ = combineLatest([this.page$, this.size$, this.filters$]).pipe(
  switchMap([page, size, filters] => this.api.list(res, { page, size, filters }));
);

```

Create Street (repeat > page size)

```

// React submit
async function submitStreet(form) {
  await api.request('post', '/streets', { body: form });
  refresh(); // triggers list refetch
}
// Vue submit
const submitStreet = async (form) => {
  await api.request('post', '/streets', { body: form });
  await refresh(); // or set reactive params to re-run watchEffect
};
// Angular submit
submitStreet(form: any) {
  this.http.post('/streets', form).pipe(
    tap(() => this.refresh())
  ).subscribe();
}

```

Edit Street

```

// React pick + submit
const [picked, setPicked] = React.useState(null);
<FetchCard onPick={setPicked} />
<OpenApiForm key={picked?.id ?? 'new'} initialData={picked} />

async function submitStreetEdit(form) {
  await api.request('put', `/streets/${form.id}`, { body: form });
  refresh();
}
// Vue pick + submit
const picked = ref(null);
<FetchCard @pick="picked = $event" />
<OpenApiForm :key="picked?.id || 'new'" :initialData="picked" />

const submitStreetEdit = async (form) => {
  await api.request('put', `/streets/${form.id}`, { body: form });
  refresh();
};
// Angular pick + submit
picked?: any;
onPick(row: any) { this.picked = row; }

submitStreetEdit(form: any) {
  this.http.put(`/streets/${form.id}`, form).pipe(
    tap(() => this.refresh())
  ).subscribe();
}

```

Create Sign (optionally filtered by streetId)

```

// React
async function submitSign(form) {
  await api.request('post', '/signs', { body: form }); // includes streetId
  refresh();
}
// Vue
const submitSign = async (form) => {
  await api.request('post', '/signs', { body: form });
  refresh();
};
// Angular
submitSign(form: any) {
  this.http.post('/signs', form).pipe(tap(() => this.refresh())).subscribe();
}

```

Edit Sign (detect move streetId A→B)

```

// React
async function submitSignEdit(form, prev) {
  const saved = await api.request('put', `/signs/${form.id}`, { body: form });
  if (prev && saved.streetId !== prev.streetId) {
    setPicked(null);
  }
  refresh();
}
// Vue
const submitSignEdit = async (form, prev) => {
  const saved = await api.request('put', `/signs/${form.id}`, { body: form });
  if (prev && saved.streetId !== prev.streetId) picked.value = null;
  refresh();
};
// Angular
submitSignEdit(form: any, prev: any) {
  this.http.put(`/signs/${form.id}`, form).pipe(
    tap((saved: any) => {
      if (prev && saved.streetId !== prev.streetId) this.picked = null;
      this.refresh();
    })
  ).subscribe();
}

```

Create Intersection

```

// React
async function submitIntersection(form) {
  await api.request('post', '/intersections', { body: form });
  refresh();
}
// Vue
const submitIntersection = async (form) => {
  await api.request('post', '/intersections', { body: form });
  refresh();
};
// Angular
submitIntersection(form: any) {
  this.http.post('/intersections', form).pipe(tap(() => this.refresh())).subscribe();
}

```

Edit Intersection

```

// React
async function submitIntersectionEdit(form) {
  await api.request('put', `/intersections/${form.id}`, { body: form });
  refresh();
}
// Vue
const submitIntersectionEdit = async (form) => {
  await api.request('put', `/intersections/${form.id}`, { body: form });
  refresh();
};
// Angular
submitIntersectionEdit(form: any) {
  this.http.put(`/intersections/${form.id}`, form).pipe(tap(() => this.refresh())).subscribe();
}

```

Reconciliation / change detection aids

```
// React
<tr key={row.id}>...</tr>
<OpenApiForm key={picked?.id ?? 'new'} ... />
// Memoize filters to avoid effect thrash
const filtersKey = useMemo(() => JSON.stringify(filters), [filters]);
// Vue
<tr v-for="row in items" :key="row.id">...</tr>
<!-- v-model keeps form in sync -->
<input v-model="form.name" />
// Angular
<tr *ngFor="let row of items; trackBy: trackById">...</tr>
trackById(index: number, row: any) { return row.id; }
// With ChangeDetectionStrategy.OnPush, update via Observables/Inputs
```